ASSIGNMENT-13

NAME: K.NAGA CHAITANYA
HALL TICKET:2303A51292
BATCH:05

#TASK 1:

PROMPT:
Refactor the given Python code to remove duplicated logic. Create a reusable function that calculates the area and perimeter of a rectangle. The function should include proper docstrings and be called for different rectangle dimensions.

CODE:

```
[3]
✓ 0s
def calculate_rectangle_area(length: float, width: float) -> float:
    """
    Calculates the area of a rectangle.

    Args:
        length (float): The length of the rectangle.
        width (float): The width of the rectangle.

    Returns:
        float: The area of the rectangle.
    """
    return length * width
def calculate_rectangle_perimeter(length: float, width: float) -> float:
    """
    Calculates the perimeter of a rectangle.

    Args:
        length (float): The length of the rectangle.
        width (float): The width of the rectangle.

    Returns:
        float: The perimeter of the rectangle.
    """
    return 2 * (length + width)
# Refactored code using the new functions
print("Area of Rectangle:", calculate_rectangle_area(5, 10))
print("Perimeter of Rectangle:", calculate_rectangle_perimeter(5, 10))

print("Area of Rectangle:", calculate_rectangle_area(7, 12))
print("Perimeter of Rectangle:", calculate_rectangle_perimeter(7, 12))

print("Area of Rectangle:", calculate_rectangle_area(10, 15))
print("Perimeter of Rectangle:", calculate_rectangle_perimeter(10, 15))
```

OUTPUT:

```
...  Area of Rectangle: 50
     Perimeter of Rectangle: 30
     Area of Rectangle: 84
     Perimeter of Rectangle: 38
     Area of Rectangle: 150
     Perimeter of Rectangle: 50
```

CODE EXPLANATION:
1) A function rectangle_properties(length, width) is created to avoid repeating the same calculations.
2) The function calculates area = length × width and perimeter = 2 × (length + width).
3) A docstring explains the purpose of the function, parameters, and return values.
4) The function returns both area and perimeter as a tuple.
5) The function is called three times with different rectangle values instead of repeating the same logic.

#TASK 2:

PROMPT:
Analyze the given Python code that uses nested loops to check whether elements exist in a list. Refactor the code to improve performance using optimized structures such as sets or list comprehensions. Provide the improved code and explain the performance difference.

CODE:
BEFORE REFACTORING:

```
[4]
✓ 0s
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]

for s in search_names:
    found = False
    for n in names:
        if s == n:
            found = True

    if found:
        print(f"{s} is in the list")
    else:
        print(f"{s} is not in the list")
```

AFTER REFACTORING:

```
[5]
✓ 0s
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]

# Convert list to set for faster lookup
names_set = set(names)

for s in search_names:
    if s in names_set:
        print(f"{s} is in the list")
    else:
        print(f"{s} is not in the list")
```

OUTPUT:

```
Charlie is in the list
Eve is not in the list
Bob is in the list
```

CODE EXPLANATION:

1) The list names is converted to a set using set(names).
2) Sets allow fast lookup operations.

VERSION	METHOD	TIME COMPLEXITY
BEFORE REFACTORING	NESTED LOOPS	O(n*m)
AFTER REFACTORING	SET LOOKUP	O(n+m)

#TASK 3:

PROMPT:
Refactor the given Python code to remove repeated price and tax calculations. Create a reusable function `calculate_total(price)` that calculates tax and total price. Ensure the function includes proper docstrings and improves code readability.

CODE:

Legacy Code(BEFORE REFACTORING):

```
[7]
✓ 0s
price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)

price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)
```

Refactored Code(AFTER REFACTORING):

```
[6]
✓ 0s
def calculate_total(price):
    """
    Calculate the total price including tax.

    Parameters:
    price (float): Original price of the product

    Returns:
    float: Total price after adding 18% tax
    """

    tax = price * 0.18
    total = price + tax
    return total


# Using the function
print("Total Price:", calculate_total(250))
print("Total Price:", calculate_total(500))
```

OUTPUT:

```
Total Price: 295.0
Total Price: 590.0
```

CODE EXPLANATION:

1) A reusable function **calculate_total(price)** is created.
2) The function is called with different price values instead of repeating code.
3) Removed duplicated calculations
4) Created reusable function
5) Impdocstrings readability and maintainability
6) Added proper documentation (docstrings)

#TASK 4:

PROMPT:
Refactor the given Python code by replacing hardcoded values with named constants. Define constants such as PI and RADIUS to improve readability and maintainability of the code.

CODE:
Legacy Code(BEFORE REFACTORING):

```
[8]
0s
print("Area of Circle:", 3.14159 * (7 ** 2))
print("Circumference of Circle:", 2 * 3.14159 * 7)
```

Refactored Code(AFTER REFACTORING):

```
[9]
✓ 0s
# Constants
PI = 3.14159
RADIUS = 7

# Calculations
area = PI * (RADIUS ** 2)
circumference = 2 * PI * RADIUS

print("Area of Circle:", area)
print("Circumference of Circle:", circumference)
```

OUTPUT:

```
...  Area of Circle: 153.93791
     Circumference of Circle: 43.98226
```

CODE EXPLANATION:
1) The hardcoded value 3.14159 is replaced with a constant PI.
2) The value 7 is replaced with a constant RADIUS.
3) Using constants makes the code clearer and easier to maintain.
4) If the radius or value of PI changes, it only needs to be updated once.

#TASK 5:

PROMPT:
Refactor the given Python code by replacing unclear variable names with descriptive ones and add comments to improve readability. The refactored code should maintain the same output.

CODE:

Legacy Code(BEFORE REFACTORING):

```
[10]
✓ 0s
a = 10
b = 20
c = a * b / 2
print(c)
```

Refactored Code(AFTER REFACTORING):

```
[11]
✓ 0s
# Base and height values
base = 10
height = 20

# Calculate the area using the formula: (base × height) / 2
triangle_area = (base * height) / 2

# Display the result
print(triangle_area)
```

OUTPUT:

```
100.0
```

CODE EXPLANATION:

1) Variables a and b are renamed to base and height for clarity.
2) Variable c is renamed to triangle_area to describe the calculation result.
3) Comments are added to explain each step of the program.
4) The logic remains the same, so the output does not change.